МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Вычислительная математика»

Тема: Метод бисекции

Студент гр. 4342 _____ Тиссен М.Ю.

Преподаватель _____ Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Цель лабораторной работы - изучить численное решение нелинейного уравнения f(x) = 0 методом бисекции (половинного деления). В ходе работы требуется отделить корень, реализовать вычисление функции варианта, подпрограммы Round и BISECT, найти корень с заданной точностью, исследовать зависимость числа итераций от точности Eps и оценить чувствительность метода к ошибкам в исходных данных при округлении значений функции с параметром Delta.

Задание.

Если найден отрезок [a,b], такой, что f (a) f (b)<0, существует точка c, в которой значение функции равно нулю, т.е. f (c)=0, c∈(a,b). Метод бисекции состоит в построении последовательности вложенных друг в друга отрезков, на концах которых функция имеет разные знаки. Каждый последующий отрезок получается делением пополам предыдущего. Процесс построения последовательности отрезков позволяет найти нуль функции f (x) (корень уравнения f (x) = 0 с любой заданной точностью.

Рассмотрим один шаг итерационного процесса. Пусть на (n-1)-м шаге найден отрезок [an-1, bn-1]⊂[a, b], такой, что f (an-1) f (bn-1)<0. Разделим его пополам точкой ξ=(an-1 +bn-1)/2 и вычислим f (ξ). Если f (ξ)=0, то ξ=(an-1+bn-1)/2- корень уравнения. Если f (ξ)≠0, то из двух половин отрезка выбирается та, на концах которой функция имеет противоположные знаки, поскольку искомый корень лежит на этой половине, т.е.

an=an-1, bn=ξ , если f (ξ) f (an-1) < 0;

an=ξ, bn= bn-1, если f (ξ) f (an-1) > 0 .

Если требуется найти корень с точностью ε, то деление пополам продолжается до тех пор, пока длина отрезка не станет меньше 2ε. Тогда координата середины отрезка есть значение корня с требуемой точностью ε.

Метод бисекции является простым и надежным методом поиска простого корня уравнения f (x) = 0 (простым называется корень x=c дифференцируемой функции f (x) , если f (c) и f ′ (c)≠0). Этот метод сходится для любых непрерывных функций f (x) , в том числе недифференцируемых. Скорость его сходимости невысока. Для достижения точности ε необходимо совершить N≈log2(b-a)/ε итераций. Это означает, что для получения каждых трех верных десятичных знаков необходимо совершить около 10 итераций.

В лабораторной работе №3 предлагается, используя программы - функции BISECT и Round из файла methods.cpp (файл заголовков metods.h, директория LIBR1), найти корень уравнения f (x) = 0 методом бисекции с заданной точностью Eps, исследовать зависимость числа 14 итераций от точности Eps

при изменении Eps от 0.1 до 0.000001, исследовать обусловленность метода (чувствительность к ошибкам в исходных данных).

Выполнение работы осуществляется по индивидуальным вариантам заданий (нелинейных уравнений), приведенным в подразделе 3.6. Номер варианта для каждого студента определяется преподавателем.

Порядок выполнения работы должен быть следующим:

1) Графически или аналитически отделить корень уравнения f (x) = 0 (т.е. найти отрезки [Left, Right], на которых функция f (x) удовлетворяет условиям теоремы Коши).

2) Составить подпрограмму вычисления функции f (x).

3) Составить головную программу, содержащую обращение к подпрограмме f(x), BISECT, Round и индикацию результатов.

4) Провести вычисления по программе. Построить график зависимости числа итераций от Eps.

5) Исследовать чувствительность метода к ошибкам в исходных данных. Ошибки в исходных данных моделировать с использованием программы Round, округляющей значения функции с заданной точностью Δ.

Вариант 18: $f(x) = e^{1/x} - ln(x)$

Выполнение работы.

1. Отделение (изоляция) корня уравнения.

Для варианта 18 рассматривается функция f(x) = e^(1/x²) − ln(x), которая определена только при x > 0. Сначала был построен график этой функции, чтобы наглядно посмотреть, где она пересекает ось Ox и на каком отрезке можно отделить корень.

По графику видно, что на отрезке [3; 4] функция меняет знак и пересекает ось Ox. Это подтверждается и вычислением значений на концах интервала: f(3) = e^(1/9) − ln(3) ≈ 0,01890678 > 0, а f(4) = e^(1/16) − ln(4) ≈ −0,32179990 < 0. Так как f(3)*f(4) < 0, на отрезке [3; 4] есть корень, и этот интервал можно взять как начальный для метода бисекции.

Если обосновывать это дополнительно через производную, то f′(x) = $-\ 2e^{(1/x^2)}/x^3\ -\ 1/x$ < 0 при x > 0, то есть функция убывает на всей области определения. Значит пересечение с осью Ox на найденном участке одно, и корень на этом интервале единственный.

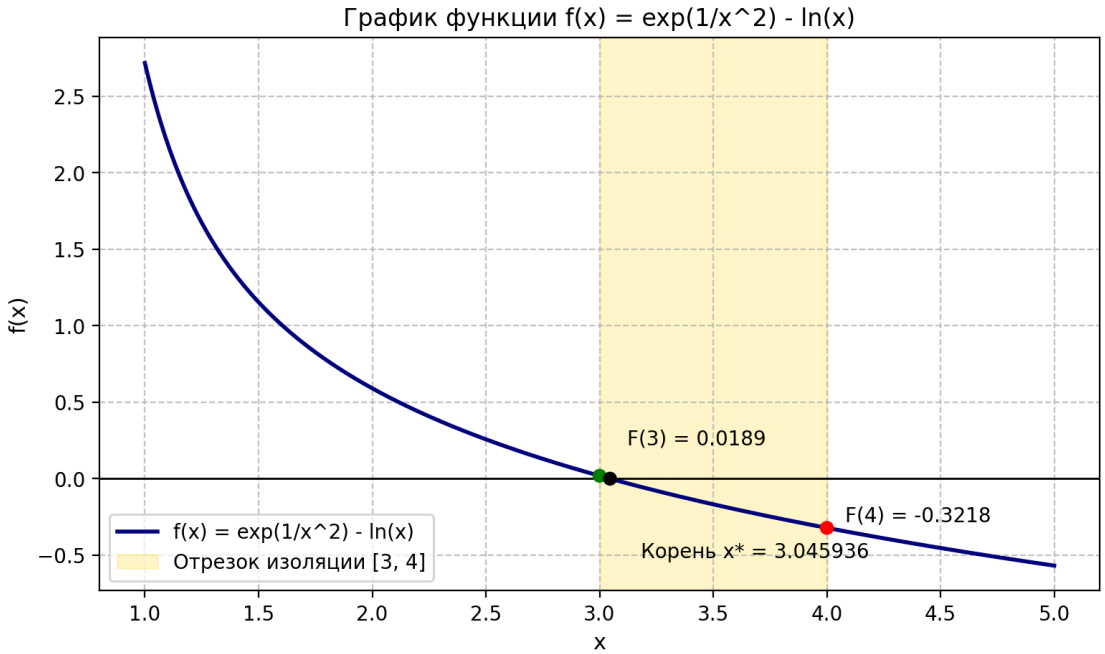

Рисунок 1 — График функции f(x) = e^(1/x²) − ln(x) и отрезок изоляции корня [3; 4].

2. Подпрограмма вычисления функции f(x).

В файле main.cpp функция варианта реализована в виде отдельной подпрограммы f(x). В ней дополнительно выполняется проверка области определения: при $x \leq 0$ программа аварийно завершается, так как выражение ln(x) для таких значений не определено. Возвращаемое значение вычисляется по формуле $f(x) = \exp(1/(x*x)) - \log(x)$.

Такая реализация соответствует условию задания и позволяет передавать f(x) в метод бисекции без дополнительных преобразований. Поскольку поиск проводится на отрезке [3; 4], вычисления происходят в допустимой области определения функции.

3. Подпрограммы Round и BISECT.

В файле methods.cpp реализована функция Round(x, Delta), которая округляет значение x до ближайшего числа, кратного Delta. Если Delta = 0, округление не выполняется. Эта функция нужна для исследования чувствительности метода к погрешностям при вычислении f(x).

Подпрограмма BISECT(Left, Right, Eps, N, Delta) реализует сам метод половинного деления. На каждом шаге находится середина текущего интервала x = (Left + Right)/2, затем вычисляется f(x) и проверяется, в какой половине отрезка сохраняется смена знака. После этого именно эта половина берётся как новый рабочий интервал.

Условие завершения в программе записано как while ((Right - Left) >= 2*Eps), поэтому работа метода заканчивается, когда длина интервала становится меньше 2Eps. Кроме того, программа может завершиться раньше, если f(Left) = 0, f(Right) = 0 или f(X) = 0. Для отрезка длины 1 теоретическая оценка числа итераций имеет вид $N \geq$ ceil(log2(1/(2Eps))), что далее сравнивается с результатами вычислений.

4. Головная программа и результаты вычислений.

Головная программа задаёт интервал [3; 4], выводит значения f(Left) и f(Right), проверяет условие смены знака и затем вызывает BISECT. Сначала

находится опорный корень с высокой точностью, после чего программа проводит серию запусков для разных значений Eps и отдельно для разных Delta. Результаты сохраняются в CSV-файлы, по которым затем строятся графики.

Eps	**Число итераций N**	**Найденный корень x***	**Теоретическая оценка**
$1 \cdot 10^{-1}$	3	3,1250000000	3
$1 \cdot 10^{-2}$	6	3,0468750000	6
$1 \cdot 10^{-3}$	9	3,0449218750	9
$1 \cdot 10^{-4}$	13	3,0460205078	13
$1 \cdot 10^{-5}$	16	3,0459442139	16
$1 \cdot 10^{-6}$	19	3,0459346771	19

Таблица 1 — Результаты вычислений при различных значениях Eps.

По результатам вычислений получен опорный корень x_ref ≈ 3,0459361048. Экспериментальные значения числа итераций совпали с теоретической оценкой: при уменьшении Eps количество шагов растёт постепенно, что хорошо видно на графике зависимости N от точности.

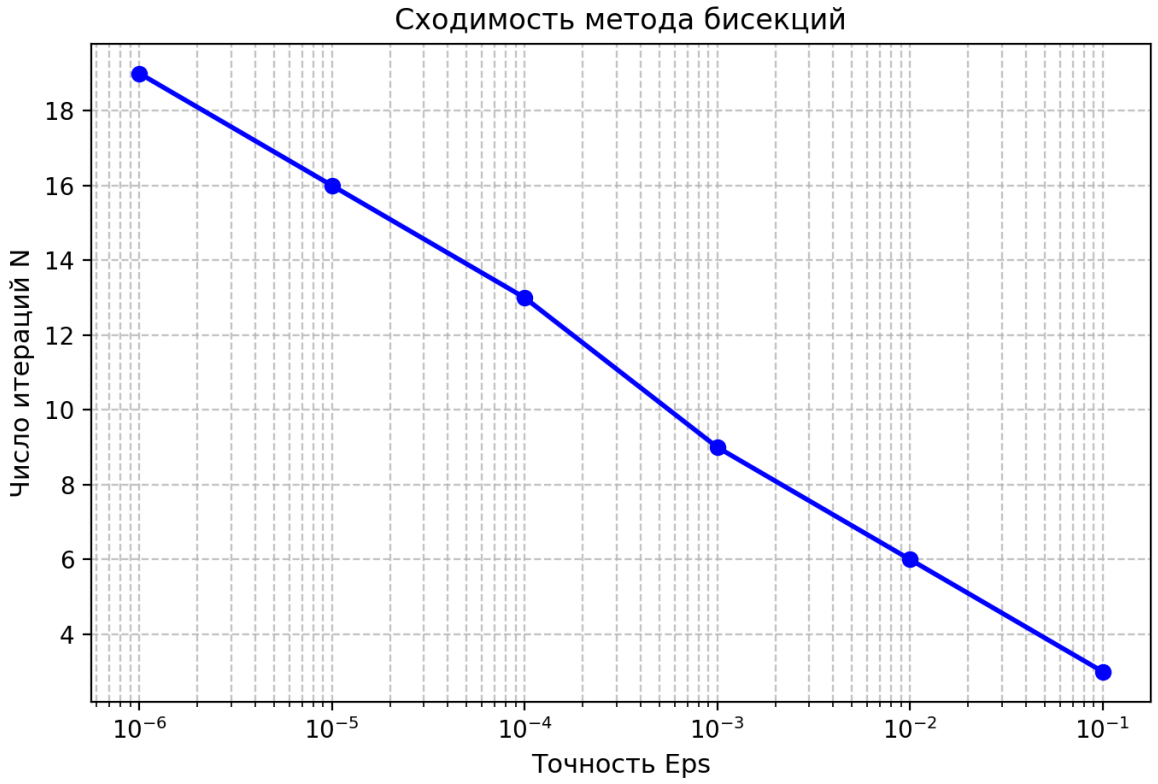

Рисунок 2 — Зависимость числа итераций метода бисекции от точности Eps.

5. Исследование чувствительности метода к ошибкам в исходных данных.

Для оценки обусловленности метода фиксировалась точность решения Eps = 10^{-6}, а значения функции округлялись с различной точностью Delta. Полученный корень сравнивался с опорным значением x_ref, найденным при очень малом Eps и без округления.

Результаты эксперимента показывают, что при грубом округлении, например при Delta = 10^{-1}, корень находится заметно хуже. Это связано с тем, что значение f(3) ≈ 0,0189 после округления становится равным 0, и программа фактически принимает левую границу интервала за корень раньше времени. При уменьшении Delta ошибка быстро снижается, что видно по графику чувствительности.

Delta	**Найденный корень Δx**	**Абсолютная ошибка**	**Комментарий**
0	3,0459346771	$1{,}4276\cdot10^{-6}$	ошибка связана с Eps = 10^{-6}
$1\cdot10^{-1}$	3,0000000000	$4{,}5936\cdot10^{-2}$	грубое округление
$1\cdot10^{-2}$	3,0468750000	$9{,}3889\cdot10^{-4}$	заметное смещение корня
$1\cdot10^{-3}$	3,0468750000	$9{,}3889\cdot10^{-4}$	результат ещё нестабилен
$1\cdot10^{-4}$	3,0458984375	$3{,}7667\cdot10^{-5}$	ошибка уменьшается
$1\cdot10^{-5}$	3,0459289551	$7{,}1496\cdot10^{-6}$	близко к эталону

Таблица 2 — Результаты исследования чувствительности к округлению.

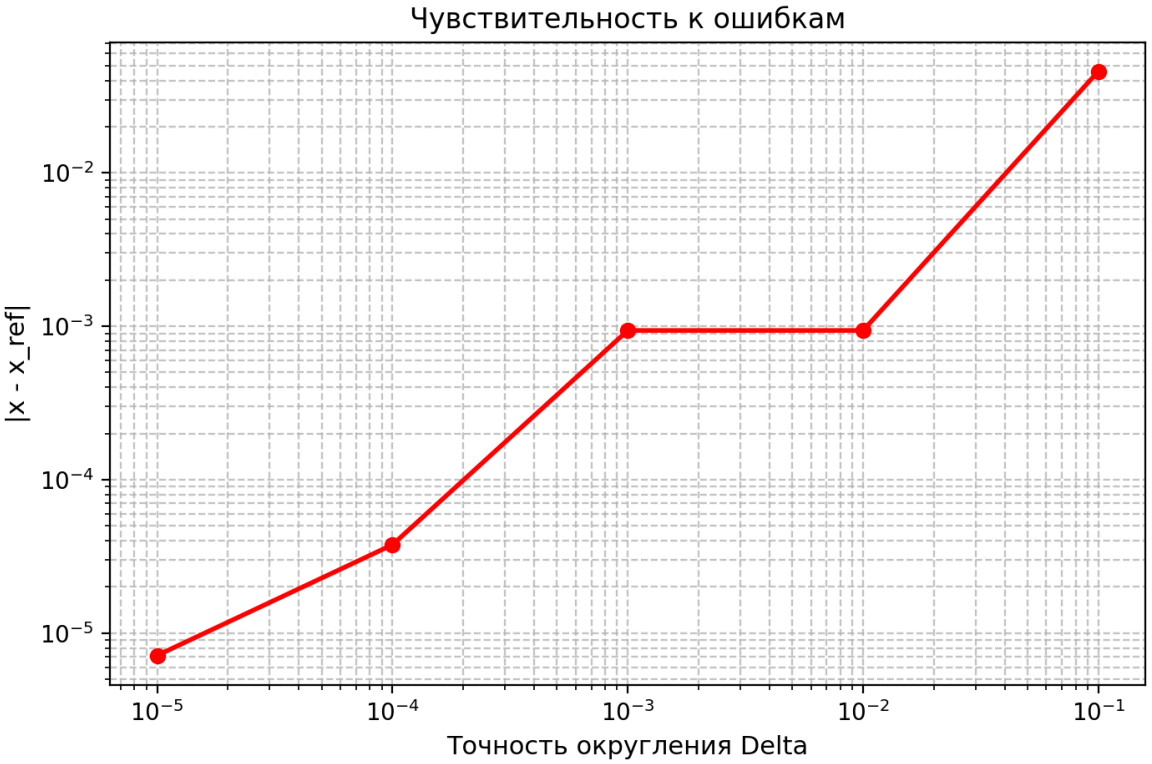

Рисунок 3 — Зависимость абсолютной ошибки |x − x_ref| от параметра Delta при Eps = 10^{-6}.

Вывод.

В ходе лабораторной работы был реализован метод бисекции для численного решения нелинейного уравнения f(x) = 0 для $f(x) = e^{1/x} - ln(x)$. Был найден интервал изоляции [3; 4], построен график функции и показано, почему на этом участке находится корень. Также было рассмотрено условие завершения алгоритма: вычисления продолжаются, пока длина текущего интервала не станет меньше 2Eps. Результаты экспериментов подтвердили корректность реализации метода, а исследование по параметру Delta показало, что точность вычисления функции существенно влияет на итоговую ошибку.

ПРИЛОЖЕНИЕ А

Название файла: methods.h

```
#ifndef METHODS_H
#define METHODS_H

// Внешняя функция F(x) - определяется в main.cpp для конкретного
варианта
extern double F(double);

// Функция округления значения X до кратного Delta
// Используется для моделирования ошибок вычислений функции
double Round(double X, double Delta);

// Метод бисекции для поиска корня F(x) = 0
// Left, Right - границы начального интервала
// Eps - требуемая точность корня
// N - число итераций (возвращается по ссылке)
// Delta - точность округления значений функции (0 = без округления)
double BISECT(double Left, double Right, double Eps, int &N, double
Delta = 0.0);

#endif
```

Название файла: methods.cpp

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

extern double F(double);

// ==============================================================
// Функция Round - округление числа до заданной точности Delta
// Округляет до ближайшего значения, кратного Delta
// Применяется для моделирования погрешностей вычислений функции
// ==============================================================
double Round(double X, double Delta)
{
    if (Delta <= 0.0) return X;  // Без округления
    if (Delta <= 1E-9) {
        printf("Неверное задание точности округления\n");
        exit(1);
    }
    // Округление до ближайшего кратного Delta
    if (X > 0.0)
        return Delta * (long)((X / Delta) + 0.5);
    else
        return Delta * (long)((X / Delta) - 0.5);
}

// ==============================================================
// Метод бисекции (половинного деления)
// Находит корень уравнения F(x) = 0 на отрезке [Left, Right]
// Delta - параметр округления значений функции (для исследования
чувствительности)
// ==============================================================
```

```
double BISECT(double Left, double Right, double Eps, int &N, double
Delta)
{
    double E = fabs(Eps) * 2.0;      // Длина интервала для остановки
    // Округляем значения функции с точностью Delta (моделирование
ошибок)
    double FLeft = Round(F(Left), Delta);
    double FRight = Round(F(Right), Delta);
    double X, Y;

    // Проверка условия теоремы Коши (разные знаки на концах)
    if (FLeft * FRight > 0.0) {
        printf("Неверное задание интервала\n");
        exit(1);
    }
    // Проверка корректности точности
    if (Eps <= 0.0) {
        printf("Неверное задание точности\n");
        exit(1);
    }

    N = 0;
    // Проверка граничных точек
    if (FLeft == 0.0) return Left;
    if (FRight == 0.0) return Right;

    // Основной цикл метода бисекции
    while ((Right - Left) >= E) {
        X = 0.5 * (Right + Left);    // Середина текущего отрезка
        // Округляем значение функции в середине (моделирование ошибок)
        Y = Round(F(X), Delta);

        if (Y == 0.0) return X;      // Точное попадание в корень

        // Выбор половины, где функция меняет знак
        if (Y * FLeft < 0.0)
            Right = X;               // Корень в левой половине
        else {
            Left = X;                // Корень в правой половине
            FLeft = Y;               // Обновляем значение функции
        }
        N++;                         // Счётчик итераций
    }
    return X;
}
```

Название файла: main.cpp

```
#include <stdio.h>
#include <math.h>
#include "methods.h"

// ==============================================================
// ВАРИАНТ 18: f(x) = e^(1/x²) - ln(x)
// Область определения: x > 0
// Интервал изоляции: [3; 4] (f(3) > 0, f(4) < 0)
// ==============================================================
double F(double x)
```

```
{
    if (x <= 0.0) {
        printf("Ошибка: x должен быть > 0\n");
        exit(1);
    }
    return exp(1.0 / (x * x)) - log(x);
}

// ==============================================================
// Головная программа
// ==============================================================
int main()
{
    // Интервал изоляции корня для Варианта 18: [3; 4]
    double Left = 3.0;
    double Right = 4.0;
    double Eps, Root, Delta, RefRoot;
    int N;

    printf("==========================================\n");
    printf("  Laboratornaya rabota #3: Metod bisekcii\n");
    printf("  Variant 18: f(x) = e^(1/x^2) - ln(x)\n");
    printf("==========================================\n\n");

    // Вывод интервала и проверка условия теоремы Коши
    printf("Interval isolyacii: [%.2f; %.2f]\n", Left, Right);
    printf("F(Left) = %.6f\n", F(Left));
    printf("F(Right) = %.6f\n", F(Right));
    printf("F(Left)*F(Right) = %.6f %s\n\n",
           F(Left)*F(Right),
                    F(Left)*F(Right)  <  0  ?  "(uslovie  vypolneno)"  :
"(OSHIBKA!)");

    // Нахождение опорного корня с высокой точностью (для сравнения)
    // Delta = 0 означает отсутствие округления (точное вычисление)
    RefRoot = BISECT(Left, Right, 1e-10, N, 0.0);
       printf("Opornyy  koren  (Eps=1e-10,  Delta=0):  %.10f  (iteraciy:
%d)\n\n", RefRoot, N);

    // ============================================================
    // Исследование зависимости числа итераций от точности Eps
    // Диапазон: от 0.1 до 0.000001 (6 значений)
    // ============================================================
    printf("Zavisimost iteraciy ot Eps:\n");
    printf("Eps\t\tIteracii\tKoren\n");
    printf("------------------------------------------\n");

    // Включаем Eps = 0.1 в массив значений
    double EpsValues[] = {0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001};
    int nEps = sizeof(EpsValues) / sizeof(EpsValues[0]);

    // Сохранение данных для построения графика
    FILE* feps = fopen("iterations_vs_eps.csv", "w");
    fprintf(feps, "Eps,Iterations,Root\n");

    for (int i = 0; i < nEps; i++) {
        Eps = EpsValues[i];
        // Delta = 0 - без округления для чистоты эксперимента
```

```
        Root = BISECT(Left, Right, Eps, N, 0.0);
        printf("%.1e\t%d\t\t%.10f\n", Eps, N, Root);
        fprintf(feps, "%e,%d,%.10f\n", Eps, N, Root);
    }
    fclose(feps);
    printf("\nDannye sohraneny: iterations_vs_eps.csv\n");

    // ==============================================================
    // Исследование чувствительности к ошибкам (параметр Delta)
    // Фиксированная точность Eps, меняем точность округления функции
    // ==============================================================
    printf("\nChuvstvitelnost metoda (Eps=1e-6):\n");
    printf("Delta\t\tKoren\t\tPogreshnost\n");
    printf("----------------------------------------\n");

    // Включаем Delta = 0 для сравнения
    double DeltaValues[] = {0.0, 0.1, 0.01, 0.001, 0.0001, 0.00001};
    int nDelta = sizeof(DeltaValues) / sizeof(DeltaValues[0]);

    FILE* fdel = fopen("sensitivity_vs_delta.csv", "w");
    fprintf(fdel, "Delta,Root,Error\n");

    for (int i = 0; i < nDelta; i++) {
        Delta = DeltaValues[i];
        // Округление применяется ВНУТРИ BISECT к значениям функции
        Root = BISECT(Left, Right, 1e-6, N, Delta);
        double Err = fabs(Root - RefRoot);
        printf("%.1e\t%.10f\t%.10f\n", Delta, Root, Err);
        fprintf(fdel, "%e,%.10f,%.10f\n", Delta, Root, Err);
    }
    fclose(fdel);
    printf("\nDannye sohraneny: sensitivity_vs_delta.csv\n");

    return 0;
}
```

Название файла: plot.py

```
import csv
import math

import matplotlib.pyplot as plt

plt.rcParams["font.family"] = "DejaVu Sans"

LEFT = 3.0
RIGHT = 4.0


def f(x):
    if x <= 0.0:
        raise ValueError("x должен быть больше 0")
    return math.exp(1.0 / (x * x)) - math.log(x)


def bisect(left, right, eps=1e-10):
    f_left = f(left)
```

```
    f_right = f(right)

    if f_left * f_right > 0.0:
        raise ValueError("неверный отрезок изоляции")

    while (right - left) >= 2.0 * eps:
        mid = 0.5 * (left + right)
        f_mid = f(mid)

        if f_mid == 0.0:
            return mid
        if f_mid * f_left < 0.0:
            right = mid
        else:
            left = mid
            f_left = f_mid

    return 0.5 * (left + right)


def finish_plot(path):
    plt.savefig(path, dpi=200, bbox_inches="tight")
    if "agg" not in plt.get_backend().lower():
        plt.show()
    plt.close()


root = bisect(LEFT, RIGHT)

# ==============================================================
# График 0: сама функция и отрезок изоляции
# ==============================================================
xs = [1.0 + i * 0.01 for i in range(401)]
ys = [f(x) for x in xs]

plt.figure(figsize=(9, 5))
plt.plot(xs, ys, color="navy", linewidth=2, label="f(x) = exp(1/x^2) -
ln(x)")
plt.axhline(0.0, color="black", linewidth=1)
plt.axvspan(LEFT, RIGHT, color="gold", alpha=0.2, label="Отрезок
изоляции [3, 4]")
plt.scatter(
    [LEFT, RIGHT, root],
    [f(LEFT), f(RIGHT), 0.0],
    color=["green", "red", "black"],
    zorder=3,
)
plt.annotate(f"F(3) = {f(LEFT):.4f}", (LEFT, f(LEFT)), xytext=(3.12,
0.22))
plt.annotate(f"F(4) = {f(RIGHT):.4f}", (RIGHT, f(RIGHT)), xytext=(4.08,
-0.28))
plt.annotate(f"Корень x* = {root:.6f}", (root, 0.0), xytext=(3.18,
-0.52))
plt.xlabel("x", fontsize=11)
plt.ylabel("f(x)", fontsize=11)
plt.title("График функции f(x) = exp(1/x^2) - ln(x)", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.7)
plt.legend()
```

```
finish_plot("function_v18.png")

# ==============================================================
# График 1: число итераций от точности Eps
# ==============================================================
eps, iters = [], []
with open("iterations_vs_eps.csv") as f_csv:
    next(f_csv)
    for row in csv.reader(f_csv):
        eps.append(float(row[0]))
        iters.append(int(row[1]))

plt.figure(figsize=(8, 5))
plt.semilogx(eps, iters, "bo-", linewidth=2, markersize=6)
plt.xlabel("Точность Eps", fontsize=11)
plt.ylabel("Число итераций N", fontsize=11)
plt.title("Сходимость метода бисекции", fontsize=12)
plt.grid(True, which="both", linestyle="--", alpha=0.7)
finish_plot("iterations_v18.png")

# ==============================================================
# График 2: погрешность корня от точности округления Delta
# ==============================================================
delta, err = [], []
with open("sensitivity_vs_delta.csv") as f_csv:
    next(f_csv)
    for row in csv.reader(f_csv):
        d = float(row[0])
        if d > 0.0:
            delta.append(d)
            err.append(float(row[2]))

plt.figure(figsize=(8, 5))
plt.loglog(delta, err, "ro-", linewidth=2, markersize=6)
plt.xlabel("Шаг округления Delta", fontsize=11)
plt.ylabel("|x - x_опорный|", fontsize=11)
plt.title("Чувствительность к ошибкам округления", fontsize=12)
plt.grid(True, which="both", linestyle="--", alpha=0.7)
finish_plot("sensitivity_v18.png")
```